Latency Arbitrage Simulator

Implementation Plan: Exchange FPGA, FPGA Sniper, PC Market Maker

This plan covers three devices. **E** is the exchange FPGA, **F** is the FPGA trader (a sniper that takes stale quotes), and **P** is the PC (a market maker that posts quotes and tries to cancel and requote after a price jump). The plan assumes your current code lets E send a packet, lets F and P reply when a value crosses a strike, and has E time the replies. It reuses those pieces. Where your design differs, adjust the matching section.

Mapping of existing pieces to the new design

Existing piece	Becomes
Packet E sends to F and P	The SIGNAL packet
Above-strike check on F and P	The staleness check (edge > strike, where strike = fee + margin)
Reply packet to E	The IOC ORDER packet
E's latency timer	E's arrival timestamp counter (foundation of everything)

The core experiment in one paragraph

P has resting quotes around fair value 100 (bid 99, ask 101). E makes fair value jump to 105 and sends SIGNAL to F and P simultaneously. F reacts in about a microsecond and sends an IOC buy at 101. P reacts in tens of microseconds and tries to cancel. If F's order reaches E first, F fills at 101, P's cancel is rejected as too late, P requotes around 105, and F sells into P's new bid at 104 for a profit of about 3 ticks per unit. Sweeping F's artificial delay shows how much speed is worth.

Part 0: Conventions (decide once, never change)

- **One clock domain for all timing.** E's free-running 64-bit counter is the only time source. F and P never need synchronized clocks, because every latency is computed as E_arrival_time minus E_signal_tx_time. At 100 MHz this gives 10 ns resolution.
- **Prices are integers in ticks** (1 tick = 0.01), so there is no floating point on the FPGA. Examples such as 101 and 105 are really 10100 and 10500.
- **Byte order:** big-endian everywhere, fixed-width fields only.
- **IDs:** order_id is a u32 owned by the sender (F uses even numbers, P uses odd, so they never collide). trader_id: E=0, F=1, P=2.

Part 1: Protocol

Every packet starts with a header: `[type u8][src u8][seq u16]`.

Type	Name	Payload
1	ORDER	order_id u32, side u8 (0=B, 1=S), tif u8 (0=rest, 1=IOC), price u32, size u16
2	FILL	order_id u32, price u32, size u16, t_exchange u64
3	BOOK	bid_px u32, bid_sz u16, ask_px u32, ask_sz u16, t_exchange u64
5	SIGNAL	round_id u16, fv u32, t_tx u64

Type	Name	Payload
6	CANCEL	order_id u32
7	ACK	order_id u32, status u8
8	CONFIG	param_id u8, value u32
9	ROUND_CTRL	cmd u8 (start/abort), round_id u16
10	LOG_RECORD	the round record (Part 7)

ACK status codes: 0 ACCEPTED, 1 FILLED, 2 NO_FILL (IOC died), 3 CANCEL_OK, 4 CANCEL_TOO_LATE, 5 BAD_ORDER.

Part 2: Exchange FPGA (E)

E is the biggest part. Build it as the blocks below.

2a. RX and timestamping

- Per-port RX parser (one for F, one for P) decodes the header and payload.
- At the moment the last byte of a packet is received, latch t_arrival = counter. Choose start-of-frame or end-of-frame once and document it.
- **Stored:** one small FIFO per port (depth about 16) holding {parsed packet, t_arrival}.

2b. Arrival arbiter

- Pulls from the two FIFOs and processes the packet with the smaller t_arrival first. On an exact tie, use a fixed priority and set a tie flag in the log.
- **Stored:** nothing beyond the FIFOs. It is a comparator and mux.

2c. Order book (simplified, since only P rests orders)

The book is two slots, not a sorted structure.

Slot	Fields (registers)
BID slot	valid, owner, order_id, price, size
ASK slot	valid, owner, order_id, price, size

A new resting bid or ask from P replaces the slot only if the slot is empty (P cancels first, so it should be). Generalizing to a small array later is a contained change.

2d. Matching engine (state machine, one packet per pass)

- **ORDER, BUY, IOC:** if ASK slot is valid and ask.price <= order.price, fill min(sizes) at **ask.price** (the resting price sets the fill), update slot sizes, emit FILL to both owners, then ACK. Otherwise ACK NO_FILL.
- **ORDER, SELL, IOC:** mirror against the BID slot.
- **ORDER, rest:** write into the matching slot, emit BOOK to both.
- **CANCEL:** if the slot's order_id matches and it is valid, clear it and send CANCEL_OK. Otherwise send CANCEL_TOO_LATE.
- After any change to the book, broadcast BOOK.

2e. Ledger (source of truth for P&L, computed in E)

- **Registers per trader (F, P):** position (signed i16), cash (signed i32).
- On each fill: buyer position += size, cash -= price*size + fee*size. Seller mirrors it.
- Round-end mark to fair value: pnl = cash + position*fv.

2f. Signal and round generator

- **Registers:** fv (u32), round_id (u16), lfsr (32-bit state), config block, round FSM state.

```
Round FSM:  IDLE -> RESET -> WAIT_QUOTES -> JUMP -> WINDOW -> RECORD -> IDLE
```

- **RESET:** clear book and ledger, set fv = 10000.
- **WAIT_QUOTES:** send ROUND_CTRL/start to P. P posts bid and ask at fv +/- spread. E waits until both slots are valid, with a timeout that aborts the round.
- **JUMP:** draw jump size from the LFSR, fv += jump, send SIGNAL to F and P, latch t_signal = counter.
- **WINDOW:** wait a fixed settle time (say 2 ms), enough for P's slow reaction to finish.
- **RECORD:** compute the summary and push it into the log buffer.
- A fixed-seed LFSR makes runs repeatable. Changing the seed via CONFIG gives a different run.

2g. CONFIG block (writable from the PC at run time, so sweeps never need resynthesis)

param_id	Name	Meaning
0	seed	LFSR seed
1 / 2	jump_min / jump_max	Jump size range (ticks)
3	spread	P's half-spread
4	fee	Per-unit fee (ticks)
5	window_us	Settle time
6	n_rounds	Rounds to run
7	signal_delay_P	Extra E-side delay before sending SIGNAL to P (cycles)

2h. Log buffer

- A BRAM ring buffer of round records (about 64 bytes each; 1,024 rounds is about 64 KB).
- Records are streamed to the PC on a LOG_DUMP request, between rounds only.

Part 3: FPGA trader (F)

Everything lives in registers. There is no memory beyond a few words.

Stored in	Item
Registers	fv_latest, ask_px, ask_sz, bid_px, bid_sz (copies from BOOK), strike (fee + margin), delay_cycles (the speed knob), next_order_id, position, avg_cost, state

Logic

- **On BOOK:** update the local book copy.
- **On SIGNAL:** store fv, wait delay_cycles (default 0), then: if fv - ask_px > strike, send BUY at ask_px, size = ask_sz, IOC. Else if bid_px - fv > strike, send SELL at bid_px, IOC.

- **On FILL (we bought):** set position and avg_cost. Wait for a bid where bid_px - avg_cost > strike, then send SELL IOC to flatten. Add a flatten timeout so F does not hold forever if P never requotes.
- **On ACK NO_FILL:** do nothing (lost the race).
- delay_cycles is a CONFIG parameter, so the latency sweep is just a PC loop.

Part 4: PC market maker (P, Python)

Stored in	Item
Python memory	fv_latest, my_bid {id, px, sz}, my_ask {id, px, sz}, next_order_id, reaction_delay (artificial extra sleep, default 0), position, cash (from FILLs, for cross-checking E)
Local log file	{event, perf_counter_ns} for diagnosis only. These timestamps are **not** used in results, because the PC clock is not synchronized with E.

- **On ROUND_CTRL start:** send ORDER rest BID at fv - spread and ORDER rest ASK at fv + spread.
- **On SIGNAL:** optional sleep, then send CANCEL for both quotes, wait for the ACKs, then post new quotes around the new fv.
- **On FILL:** update position and cash, then requote if needed.
- The PC will have real jitter from the OS and network stack. Do not hide it. Use it as the realistic latency distribution. Pin the process to a core and disable power saving so the jitter is your own and not noise.

Part 5: Experiment control (PC script)

A separate Python controller (it can share a process with P) that:

- Sends CONFIG parameters, then ROUND_CTRL start for N rounds.
- Pulls the log buffer between rounds.
- Loops over the sweep: for each F_delay in [0, 1, 2, 5, 10, 20, 50, 100, 200] microseconds, runs about 1,000 rounds and saves a CSV named by its parameters.

Part 6: Calibration (before trusting any result)

- **Link latency per device:** use your existing packet-and-reply timing to measure E->F->E and E->P->E baseline round trips. Subtract known baselines, or at least report them, so cable and transceiver differences are not mistaken for strategy differences.
- **Signal fan-out skew:** log the actual TX time to each port. If F's port goes out later than P's, correct for it with signal_delay_P.
- **PC jitter profile:** 10,000 echo trials, report median, p99, and max.

Part 7: Log record (per round, stored in E's BRAM)

```
round_id u16, jump_size i16, fv_after u32,
t_signal u64,
F_arrival u64 (0 if no order), P_cancel_arrival u64, P_requote_arrival u64,
winner u8 (0 none, 1 F, 2 P-cancelled-first),
tie_flag u8,
F_fill_px u32, F_fill_sz u16, F_exit_px u32,
pnl_F i32, pnl_P i32
```

Derived results: win rate vs. F delay, F's reaction latency (F_arrival - t_signal), P's cancel latency, P&L per round by role, and win rate vs. jump size.

Part 8: Milestones with pass/fail tests

#	Build	Test that proves it works
1	Common header, 64-bit counter, RX timestamping	Send 1,000 packets from F and P. Timestamps are monotonic and match your existing latency numbers.
2	Book slots, resting orders, BOOK broadcast	PC posts a bid and an ask. Both F and P see identical BOOK packets.
3	Matching, IOC, fills, cancels, ledger	Scripted tests: buy fills at resting price, partial fills, cancel-after-fill returns TOO_LATE, ledger sums to zero across both traders (minus fees).
4	Signal generator, round FSM, config	100 rounds with a fixed seed produce identical jump sequences on two runs.
5	F strategy with delay knob	At delay=0, F wins about 100% of rounds. At very large delay, about 0%.
6	P market maker with cancel and requote	P's quotes follow fv across rounds. CANCEL_TOO_LATE appears when F wins.
7	Log buffer, dump, analysis script	Plot win rate vs. F delay. The cliff lands near P's measured reaction time.
8	Calibration and sweeps	Reproducible curves with error bars from repeated seeds.

Risk notes

- Milestone 3 (matching) is where most bugs will live. Test it in a simulation testbench before putting it on hardware.
- If each device has only one link, decide early whether logs go out over the same link or a separate UART. A shared link can perturb timing during a round, so dump logs between rounds only.
- Keep the 2-slot book until everything works. Generalizing early is the main source of scope creep.

Future phases

- **Phase 2:** let F also quote, so P is sniped and competed against. Shows a fast maker is sniped less and captures more spread. Needs a price-time priority rule.
- **Phase 3:** swap roles, putting the maker on the FPGA and the sniper on the PC. The PC sniper rarely wins, which reinforces the value of speed.
- **Realism upgrades:** leader/follower instruments so F must infer fair value, latency distributions with tails, fees and tick sizes, and spread adaptation by P after repeated sniping.

Pitch caveat: this is a toy model. Real latency arbitrage involves queue position, fee structures, and co-location, and the edge is much thinner than a clean 101 to 104 flip. Saying so makes the project more credible.